

Service Discovery

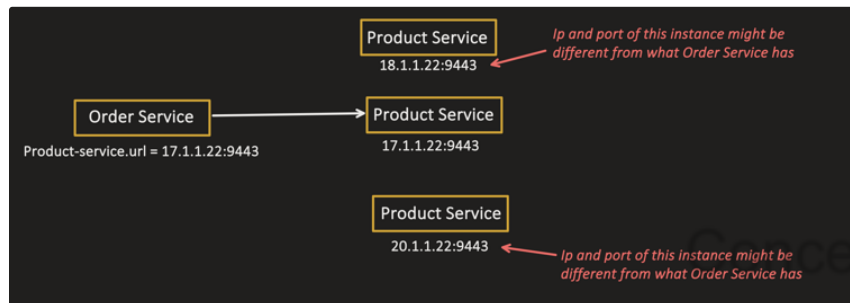
- In Microservices, services or component instances are created and deleted dynamically.
- We can not hardcode the URL of an instance, its not scalable and feasible.

[https://
drive.google.com/
file/](https://drive.google.com/file/)

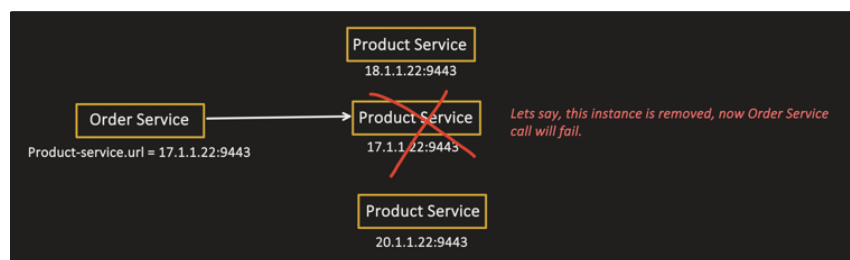


In above example:

- Order service has hardcoded the URL of the Product service and its ok till we have only 1 instance of Product service.
- But in case, when we have multiple Product service instances then....



Also, possible that, particular instance is removed



Now in above example, where multiple instances of Product service is present and Order Service has hardcoded the URL of the Product service, the problem it might face is:

- Single point of failure:
If hardcoded instance of Product Service goes down, Order service will not be able to communicate with any other instance.
- No Load Balancing:
Only one instance of Product Service get overburdened while other instances remain idle.
- Tight Coupling:
Because of hardcoded URL in Order Service, there is tight coupling between Product and Order Service, as without

updating the Order Service its not possible to change or move Product Service.

- Difficulty in testing:

Different environment (say production, QA, dev) might uses different URL, which require frequent changes in config and not only cause difficulty in testing but prone to error too.

Solution for above problem is: **SERVICE DISCOVERY (like Eureka)**

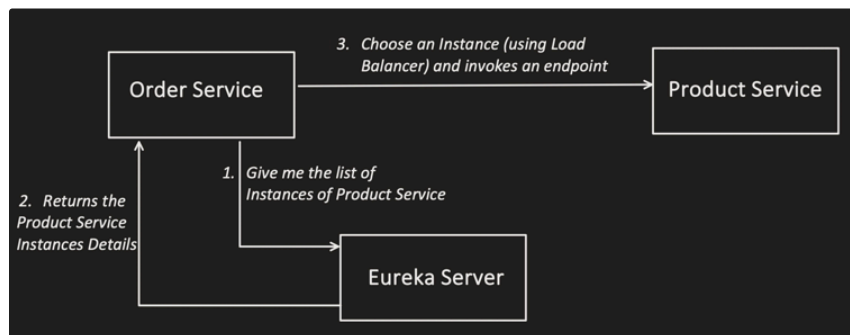
Eureka Server:

- Act like a phonebook.
- Has all the instances info of all the registered clients like:
 - Service name
 - Instance id
 - IP
 - Port number
 - Health status etc.

Eureka Client:

- Register itself with the Server.
- Discovers an instance of other service via Eureka Server

Concept && Coding



Lets first set up Eureka Server Application:

Go to Spring Initializer (start.spring.io)

Project

☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Java
☐ Kotlin
☐ Groovy

Language

☒ Maven

Spring Boot

☐ 4.0.0 (SNAPSHOT)
☐ 3.5.1 (SNAPSHOT)
☐ 3.5.0
☐ 3.4.7 (SNAPSHOT)
☒ 3.4.6
☐ 3.3.13 (SNAPSHOT)
☐ 3.3.12

Project Metadata

Group

com.example

Artifact

EurekaServer

Name

EurekaServer

Description

Learning of Service Discovery

Package name

com.example.EurekaServer

Packaging

☒ Jar
☐ War

Java

☐ 24
☒ 21
☐ 17

Dependencies

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. default embedded container.

1. pom.xml

```

1 <dependency>
2 <groupId>org.springframework.cloud</groupId>
3 <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
4 </dependency>
5

```

Version for "spring-cloud-starter-netflix-eureka-server" will be automatically resolved by below dependency management.

```

1 <dependencyManagement>
2 <dependencies>
3 <dependency>
4 <groupId>org.springframework.cloud</groupId>
5 <artifactId>spring-cloud-dependencies</artifactId>
6 <version>2023.0.1</version>
7 <type>pom</type>
8 <scope>import</scope>
9 </dependency>
10 </dependencies>
11 </dependencyManagement>
12

```

Concept && Coding

2. Enable Eureka Server functionality

```

1 @SpringBootApplication
2 /*
3 Tells Spring boot to create necessary beans,
4 which is required for Eureka Server like:
5 - EurekaController
6 - Dashboard etc..
7 */
8 @EnableEurekaServer
9 public class EurekaServerApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EurekaServerApplication.class, args);
13     }
14 }

```

3. application.properties

```

1 spring.application.name=eureka-server
2 server.port=8761
3

```

```

4 # Since it's a server, we don't want it to register and
5 # also don't want to fetch the instances details
6 eureka.client.register-with-eureka=false
7 eureka.client.fetch-registry=false
8

```

Lets start the Server and see the dashboard:

```

2025-06-12T15:22:06.250+05:30 INFO 43767 --- [eureka-server] [
main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint(s) beneath base path '/actua
2025-06-12T15:22:06.270+05:30 INFO 43767 --- [eureka-server] [
main] o.s.c.n.e.s.EurekaServiceRegistry : Registering application EUREKA-SERVER with eureka
2025-06-12T15:22:06.274+05:30 INFO 43767 --- [eureka-server] [
Thread-9] o.s.c.n.e.s.server.EurekaServerBootstrap : isAses returned false
2025-06-12T15:22:06.274+05:30 INFO 43767 --- [eureka-server] [
Thread-9] o.s.c.n.e.s.server.EurekaServerBootstrap : Initialized server context
2025-06-12T15:22:06.274+05:30 INFO 43767 --- [eureka-server] [
Thread-9] o.s.c.n.e.r.PeerAwareInstanceRegistryImpl : Got 1 instances from neighboring DS node
2025-06-12T15:22:06.274+05:30 INFO 43767 --- [eureka-server] [
Thread-9] o.s.c.n.e.r.PeerAwareInstanceRegistryImpl : Renew threshold is: 1
2025-06-12T15:22:06.274+05:30 INFO 43767 --- [eureka-server] [
Thread-9] o.s.c.n.e.r.PeerAwareInstanceRegistryImpl : Changing status to UP
2025-06-12T15:22:06.276+05:30 INFO 43767 --- [eureka-server] [
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8761 (http) with context
2025-06-12T15:22:06.276+05:30 INFO 43767 --- [eureka-server] [
main] .s.c.n.e.s.EurekaAutoServiceRegistration : Updating port to 8761
2025-06-12T15:22:06.278+05:30 INFO 43767 --- [eureka-server] [
Thread-9] e.s.EurekaServerInitializerConfiguration : Started Eureka Server
2025-06-12T15:22:06.291+05:30 INFO 43767 --- [eureka-server] [
main] c.e.E.EurekaServerApplication : Started EurekaServerApplication in 1.513 seconds

```

<http://localhost:8761>

The screenshot shows the Spring Eureka dashboard. The top navigation bar includes 'HOME' and 'LAST 1000 SINCE STARTUP'. The 'System Status' section displays the following information:

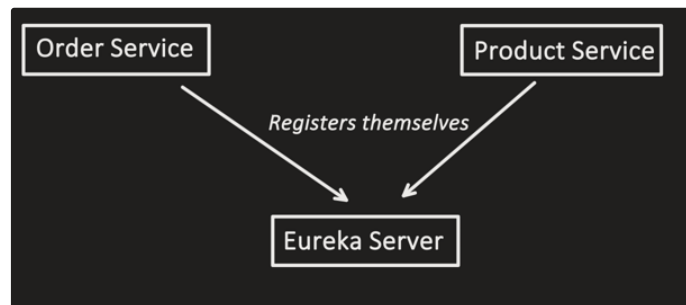
Environment	test	Current time	2025-06-12T15:22:44 +0530
Data center	default	Uptime	00:00
		Lease expiration enabled	false
		Renews threshold	1
		Renews (last min)	0

The 'DS Replicas' section shows a single replica at 'localhost'.

The 'Instances currently registered with Eureka' section shows a table with columns: Application, AMIs, Availability Zones, and Status. It currently shows 'No instances available'.

The 'General Info' section is also visible at the bottom.

Now lets set up Eureka Client Application:



Product Service

1. pom.xml

```

1 <dependency>
2 <groupId>org.springframework.cloud</groupId>
3 <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
4 </dependency>
5

```

Version for "*spring-cloud-starter-netflix-eureka-client*" will be automatically resolved by below dependency management.

```
1 <dependencyManagement>
2 <dependencies>
3 <dependency>
4 <groupId>org.springframework.cloud</groupId>
5 <artifactId>spring-cloud-dependencies</artifactId>
6 <version>2023.0.1</version>
7 <type>pom</type>
8 <scope>import</scope>
9 </dependency>
10 </dependencies>
11 </dependencyManagement>
12
```

2. application.properties

```
1 server.port=8082
2 spring.application.name=product-service
3
4 # path of the Eureka Server
5 eureka.client.service-url.defaultZone=http://localhost:8761/eureka
6
7 # by-default values are true only, so we can even skip below configs
8 eureka.client.register-with-eureka=true
9 eureka.client.fetch-registry=true
10
```

Lets start the Product Service Server and see the dashboard again:

Concept && Coding

```
2025-06-12T15:42:49.204+05:30 INFO 43993 --- [product-service] [main] com.netflix.discovery.DiscoveryClient : Starting heartbeat executor: renew interval is: 30
2025-06-12T15:42:49.264+05:30 INFO 43993 --- [product-service] [main] c.n.discovery.InstanceInfoReplicator : InstanceInfoReplicator onDemand update allowed rate per sec
2025-06-12T15:42:49.267+05:30 INFO 43993 --- [product-service] [main] com.netflix.discovery.DiscoveryClient : Discovery client initialized at timestamp: 1749723169207 s
2025-06-12T15:42:49.268+05:30 INFO 43993 --- [product-service] [main] o.s.c.n.s.EurekaServiceRegistry : Registering application PRODUCT-SERVICE with eureka with st
2025-06-12T15:42:49.269+05:30 INFO 43993 --- [product-service] [main] com.netflix.discovery.DiscoveryClient : See local status change event StatusChangeEvent [timestamp
2025-06-12T15:42:49.269+05:30 INFO 43993 --- [product-service] [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient_PRODUCT-SERVICE/192.168.1.101:product-serv
2025-06-12T15:42:49.218+05:30 INFO 43993 --- [product-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path ''
2025-06-12T15:42:49.218+05:30 INFO 43993 --- [product-service] [main] i.c.n.s.o.EurekaAutodiscoveryRegistration : updating port to 8082
2025-06-12T15:42:49.225+05:30 INFO 43993 --- [product-service] [main] o.s.c.p.ProductServiceApplication : Started ProductServiceApplication in 1.014 seconds (process
2025-06-12T15:42:49.259+05:30 INFO 43993 --- [product-service] [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient_PRODUCT-SERVICE/192.168.1.101:product-serv
```

The screenshot shows the Spring Eureka dashboard in a web browser. The dashboard has a dark header with the 'spring Eureka' logo and navigation links for 'HOME' and 'LAST 1000 SINCE STARTUP'. Below the header, there's a 'System Status' section with a table showing environment (test), data center (default), current time (2025-06-12T15:45:35 +0530), uptime (00:00), lease expiration enabled (false), renew threshold (3), and renew last min (0). Below this is a 'DS Replicas' section showing 'localhost'. The 'Instances currently registered with Eureka' section contains a table with columns: Application, AMIs, Availability Zones, and Status. A single instance 'PRODUCT-SERVICE' is listed with 'n/a (1)' for AMIs, '(1)' for Availability Zones, and 'UP (1) - 192.168.1.101:product-service:8082' for Status. The status cell is highlighted with a red box. At the bottom, there's a 'General Info' section.

Application	AMIs	Availability Zones	Status
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.101:product-service:8082

Similarly for Order Service, we can register it with Eureka Server

The screenshot shows the Spring Eureka dashboard. The 'System Status' section displays environment details (test, default) and system metrics (uptime, lease expiration, renewals). The 'Instances currently registered with Eureka' table lists two services: ORDER-SERVICE and PRODUCT-SERVICE, both with one instance up at the specified IP addresses. The 'DS Replicas' section shows 'localhost'.

System Status			
Environment	test	Current time	2025-06-12T15:48:29 +0530
Data center	default	Uptime	00:03
		Lease expiration enabled	false
		Renews threshold	5
		Renews (last min)	4

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.101:order-service-8081
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.101:product-service-8082

Now lets see, how Order Service can invoke Product Service:

Using RestTemplate:

Without Service Discovery:

```

1 //specifically mentioning the URL
2 public void callProductAPI(String id) {
3
4     RestTemplate restTemplate = new RestTemplate();
5     String response = restTemplate.getForObject(
6         "http://localhost:8082/products/" + id,
7         String.class
8     );
9
10    System.out.println("Response from Product api call is: " +
11        response);
12 }

```

With Service Discovery

```

1 Import org.springframework.cloud.client.ServiceInstance;
2 Import org.springframework.cloud.client.discovery.DiscoveryClient;
3
4 @Autowired
5 DiscoveryClient discoveryClient;
6
7 public void callProductAPI(String id) {
8
9     RestTemplate restTemplate = new RestTemplate();
10
11    List<ServiceInstance> instances =
12        discoveryClient.getInstances("product-service");
13
14    URI uri = instances.get(0).getUri();
15
16    String response = restTemplate.getForObject(
17        uri + "/products/" + id,
18        String.class
19    );
20
21    System.out.println("Response from Product api call is: " +
22        response);
23 }

```

```
22 }  
23
```

So with RestTemplate, load balancing (choosing an instance for the product service) logic need to be handled.

Using FeignClient:

So with FeignClient, load balancing is handled automatically and by the framework.

So we need to provide the Load Balancer dependency too, apart from *"spring-cloud-starter-netflix-eureka-client"*

```
1 <dependency>  
2 <groupId>org.springframework.cloud</groupId>  
3 <artifactId>spring-cloud-starter-loadbalancer</artifactId>  
4 </dependency>  
5
```

Earlier without Service Discovery:

```
1 @FeignClient(  
2     name = "product-service",  
3     url = "${feign.client.product-service.url}"  
4 )  
5 public interface ProductClient {  
6  
7     @GetMapping("/products/{id}")  
8     String getProductById(@PathVariable("id") String id);  
9 }
```

Concept && Coding

With Service Discovery:

```
1 @FeignClient(name = "product-service")  
2 public interface ProductClient {  
3  
4     @GetMapping("/products/{id}")  
5     String getProductById(@PathVariable("id") String id);  
6 }
```

As a curious engineer, few questions comes to our mind:

Service Registration doubts:

1. How does Eureka Server know whether a client is UP or DOWN?
2. Where and how the data is stored?
3. What if Eureka Server itself goes down? Is it a single point of failure?

Discovery Doubts:

1. It can cause latency issue, as each call now required 2 hops, first it has to invoke Eureka Server and then the actual call.
2. What if the local cache is stale? Can this lead to calling a dead instance?

Lets try to find an answer one by one:

1. How does Eureka Server know whether a client is UP or DOWN?

Through Client de-registration request:

When client application is gracefully shut down, then eureka client sends the de-registration request to Eureka Server, it mark client status as DOWN.

System Status			
Environment	test	Current time	2025-06-13T11:56:15 +0530
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	3
		Renews (last min)	0
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.0.37:product-service:8082

Client(product-service) application logs :

```
product-service [ main] com.netflix.discovery.DiscoveryClient : Getting all instance registry info from the eureka server
product-service [ main] com.netflix.discovery.DiscoveryClient : The response status is 200
product-service [ main] com.netflix.discovery.DiscoveryClient : Starting heartbeat executor: renew interval is: 30
product-service [ main] com.netflix.discovery.DiscoveryClient : InstanceInfoReplicator onDemand update allowed rate per min is 4
product-service [ main] com.netflix.discovery.DiscoveryClient : Discovery Client initialized at timestamp 1749794212395 with initial instances count: 0
product-service [ main] com.netflix.discovery.DiscoveryClient : Registering application PRODUCT-SERVICE with eureka with status UP
product-service [ main] com.netflix.discovery.DiscoveryClient : See local status change event StatusChangeEvent [timestamp=1749794212395, current=UP, previous=STARTING]
product-service [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient.PRODUCT-SERVICE/192.168.0.37:product-service:8082: registering service...
product-service [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path ''
product-service [ main] o.s.b.w.eureka.EurekaAutoServiceRegistration : Updating port to 8082
product-service [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient.PRODUCT-SERVICE/192.168.0.37:product-service:8082 - registration status: 204
product-service [ main] com.netflix.discovery.DiscoveryClient : Started ProductServiceApplication in 0.974 seconds (process running for 1.113)
product-service [infoShutdownHook-0] o.s.c.n.e.s.EurekaServiceRegistry : Unregistering application PRODUCT-SERVICE with eureka with status DOWN
product-service [infoReplicator-0] com.netflix.discovery.DiscoveryClient : See local status change event StatusChangeEvent [timestamp=1749794220419, current=DOWN, previous=UP]
product-service [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient.PRODUCT-SERVICE/192.168.0.37:product-service:8082: registering service...
product-service [infoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient.PRODUCT-SERVICE/192.168.0.37:product-service:8082 - registration status: 204
product-service [infoShutdownHook-0] com.netflix.discovery.DiscoveryClient : Shutting down DiscoveryClient ...
product-service [infoShutdownHook-0] com.netflix.discovery.DiscoveryClient : Unregistering ...
```

Server (Eureka-server) application logs :

```
[eureka-server] [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 5ms
[eureka-server] [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 5ms
[eureka-server] [info-8761-exec-0] c.n.e.registry.AbstractInstanceRegistry : Registered instance PRODUCT-SERVICE/192.168.0.37:product-service:8082 with status UP (replication=false)
[eureka-server] [info-8761-exec-0] c.n.e.registry.AbstractInstanceRegistry : Registered instance PRODUCT-SERVICE/192.168.0.37:product-service:8082 with status DOWN (replication=true)
```

System Status			
Environment	test	Current time	2025-06-13T11:56:33 +0530
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	3
		Renews (last min)	0
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
PRODUCT-SERVICE	n/a (1)	(1)	DOWN (1) - 192.168.0.37:product-service:8082

Through Client Heart Beat

Eureka Client periodically sends Heart Beat to the Eureka Server.

Lets say, if client shut down without sending any de-registration request (bcoz of Network issue).

then Eureka server wait for the heart beat from Client for a particular interval (decided at the time of registration) and if no Heart Beat received within that time interval, it remove the client instance itself.

Client(product-service) ***application.properties:***

```
1 server.port=8082
2 spring.application.name=product-service
3
4 # path of the Eureka Server
5 eureka.client.service-url.defaultZone=http://localhost:8761/eureka
6
7 # by-default values are true only, so we can even skip below configs
8 eureka.client.register-with-eureka=true
9 eureka.client.fetch-registry=true
10
11 # every 60 seconds client will send the heart beat to server
12 eureka.instance.lease-renewal-interval-in-seconds=60
13
14 # telling server to wait this much time, if no heart beat received in
   this time, then remove me.
15 # for testing, kept it less than lease-renewal-interval
16 eureka.instance.lease-expiration-duration-in-seconds=5
17
```

eureka.instance.lease-renewal-interval-in-seconds: *After this much seconds, client will send the new heart beat to server. Default time is 30sec*

Concept & Coding

eureka.instance.lease-expiration-duration-in-seconds: *Telling server to wait only this much time for the heart beat. After that, you can remove it from the list. Default time is 90sec*

Eureka Server ***application.properties:***

```
1 spring.application.name=eureka-server
2 server.port=8761
3
4 # Since it's a server, we don't want it to register and
5 # also don't want to fetch the instances details
6 eureka.client.register-with-eureka=false
7 eureka.client.fetch-registry=false
8
9 # by default Server do not remove the client, even heart beat is
   missed.
10 # so we need to turn this OFF and allow the server to remove if client
   heart beat missed.
11 eureka.server.enable-self-preservation=false
12
13 # how frequently eviction task runs
14 eureka.server.eviction-interval-timer-in-ms=6000
```

System Status			
Environment	test	Current time	2025-06-13T12:32:29 +0530
Data center	default	Uptime	00:04
		Lease expiration enabled	true
		Renews threshold	3
		Renews (last min)	0
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.0.37:product-service:8082

Because in above config:

- Client will send the heart beat after - 60seconds
- Server will wait for the heart beat only for - 5 seconds

So server, removed the client instance from the list, even Client application is running and up.

System Status			
Environment	test	Current time	2025-06-13T12:32:39 +0530
Data center	default	Uptime	00:04
		Lease expiration enabled	true
		Renews threshold	1
		Renews (last min)	0
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
No instances available			

2. How does Eureka Server stores the data

Eureka Server only stores the data in-memory : Map<String, Lease<InstanceInfo>>

- Key : *appName/instanceId*

ex:

PRODUCT-SERVICE/192.157.2.27:product-service:8082

- value: *InstanceInfo* object

- Instance ID
- App name
- IP address
- Host name
- Port
- Status (UP, DOWN)
- Last renewed timestamp
- Lease duration

Etc.

3. What if Eureka Server itself goes down? Is it a single point of failure?

Yes, a Single Eureka Server is a Single point of failure, if it goes down.

- Usually, 3 nodes cluster is used.

Say, we have 3 Eureka Server on different machine or container:

- eureka-1 at port 8761
- eureka-2 at port 8762
- eureka-3 at port 8763

Now, each Server is a client too. As they have to register themselves, fetch the registry, replicate changes.

application.properties for eureka server 1:

```
spring.application.name=eureka-server
server.port=8761
eureka.instance.hostname=localhost

eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.client.service-url.defaultZone= http://localhost:8762/eureka/, http://localhost:8763/eureka/
```

application.properties for eureka server 2:

```
spring.application.name=eureka-server
server.port=8762
eureka.instance.hostname=localhost

eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.client.service-url.defaultZone= http://localhost:8761/eureka/, http://localhost:8763/eureka/
```

application.properties for eureka server 3:

```
spring.application.name=eureka-server
server.port=8763
eureka.instance.hostname=localhost

eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.client.service-url.defaultZone= http://localhost:8761/eureka/, http://localhost:8762/eureka/
```

And in Client application.properties:

```
eureka.client.service-url.defaultZone= http://eureka-1:8761/eureka, http://eureka-2:8762/eureka, http://eureka-3:8763/eureka
```

Concept && Coding

Now even if 1 server is down, it will not impact the client availability.

4. **It can cause latency issue, as each call now required 2 hops, first it has to invoke Eureka Server and then the actual call.**

- Eureka Server does not get called for every request.
- At startup, Client say (order-service) fetch the registry
eureka.client.fetch-registry=true
- And Cache it locally, all future request, uses this local copy to find the instance.

we can control this with config like below, after every 30 seconds client will refresh its cache copy:

eureka.client.registry-fetch-interval-seconds=30

5. **What if the local cache is stale? Can this lead to calling a dead instance?**

It's a valid scenario and you can say it's a trade off.

